OpenVAF — Verilog-A LRM Compliance

Clause-by-clause coverage of the Verilog-A language (VAMS LRM 2023, Annex C)

Ngspice + OpenVAF Enhancements project

July 2026

Contents

OpenVAF — Verilog-A LRM Compliance

How the openvaf-r compiler in this repository covers the Verilog-A language, clause by clause against the Verilog-AMS Language Reference Manual (the 2023 edition, `docs/VAMS-LRM-2023.pdf`; Verilog-A is its Annex C analog-only subset). Every claim in this document is backed by a committed, executable check: the code examples are drawn from the verify suites under `examples/`, which compile each construct with the committed compiler, simulate it with the committed ngspice, and compare the numbers against closed-form expectations on every regression run.

Status marks used throughout:

Mark	Meaning
✓	fully implemented and runtime-verified
⚠	implemented with LRM-sanctioned fallback semantics, or a documented deviation
×	out of scope: a Verilog-AMS (mixed-signal) feature outside Annex C

A one-page summary table closes the document. Companion references: the User Handbook (task-oriented), the openvaf change report (where each capability was implemented and why), and the per-enhancement write-ups in `enhancements_doc/`.

---

1. Compilation model and scope (LRM 1, Annex C)

OpenVAF compiles one Verilog-A source file (with its `` `include `` closure) into an OSDI shared object: analytic residuals and Jacobians are generated by automatic differentiation, so every construct below is "supported" only if it produces correct derivatives as well as correct values — a stricter bar than parsing. Hierarchy is resolved by a compile-time elaboration pass (§7), after which the pipeline sees a single flat module.

Annex C draws the language boundary this project targets: the analog subset. Digital and mixed-signal constructs (`reg`/`wire` logic, digital `always`/`initial`, connect modules, `` `default_discipline ``, discrete-domain disciplines) are × out of scope by definition, and the compiler rejects them with located diagnostics rather than accepting them silently — e.g. `domain discrete` on a discipline with natures is a proper two-label error citing LRM 3.6.2.2.

2. Lexical conventions (LRM 2) — ✓

Identifiers, including escaped identifiers (LRM 2.7.1):

```
electrical \node-1a ;          // any printable characters until whitespace
parameter real \my.gain = 2.0;
```

Numbers (LRM 2.5): decimal integers, reals with exponents, SI scale suffixes (`1k`, `2.5u`, `10n`), and based integer literals in all four bases with optional size and sign, underscores as separators:

```
integer a, b, c, d;
analog begin
    a = 8'hFF;          // sized hex          -> 255
    b = 'sb1010_1100;   // signed binary with separators
    c = 'o17;           // octal              -> 15
    d = 4'd12;          // sized decimal
end
```

The don't-care digits `x/z/?` are additionally accepted in the power-of-two bases when the literal is a `casex/casez` item (§5.4); anywhere else they are a located error rather than silent zeros.

Strings (LRM 2.6.3) with the full escape set — `\n`, `\t`, `\\`, `\"`, and octal `\ddd` — processed by a single-pass unescaper (a sequential-replacement implementation corrupted overlapping escapes and was rewritten). Comments both styles, including the edge case of a `//` comment at end-of-file without a trailing newline (formerly a lexer hang). Attributes `(* ... *)` parse everywhere the LRM allows and carry simulator-facing metadata (§9.6).

3. Data types (LRM 3)

3.1 Value types — ✓

`integer` (32-bit), `real` (IEEE double), and `string` variables, with genuine persistence across evaluations for both numeric types (a Verilog-A module variable keeps its value between solver calls, the foundation of event counters and iterative models):

```
integer count;            // persists across evaluations
analog begin
    @(cross(V(in) - 0.5, +1))
        count = count + 1; // genuinely accumulates
end
```

Uninitialized strings default to `""`; declaration initializers work on scalars and (multi-dimensional) arrays, evaluating once at setup when parameter-only:

```
real x = 2.0 * gain;
real taps[0:3] = '{0.1, 0.2, 0.3, 0.4};
```

3.2 Parameters (LRM 3.4) — ✓

Ranges and exclusions enforced on user-given values; `localparam` genuinely non-overridable; `aliasparam` including `$param_given` through the alias; string parameters (usable in `case` dispatch); array parameters of any dimensionality, expanded to per-element OSDI parameters that are individually overridable from SPICE:

```
parameter real r = 1k from (0:inf) exclude 1e6;
parameter integer sel = 0 from [0:15];
localparam real g = 1.0 / r;                // tracks r, not overridable
parameter real w[0:1][0:2] = '{'{1,2,3}, '{4,5,6}};
// SPICE: .model m dev(w[1][2]=9.5)
```

One deliberate reading of the LRM, matching industry practice (the CMC convention): a parameter's default is exempt from its own range — compact models use out-of-range defaults as "feature disabled" markers — while every user-given value is fully validated. ⚠ documented in the handbook's limitations chapter.

Block-scoped parameters (LRM 6.3): a `parameter/localparam` declared inside a named `begin: label` block is a compile-time constant local to that block, read hierarchically and derivable from the module's parameters (E-87):

```
analog begin
   begin: s
      parameter real g2 = gain * gain;  // block param, from a module param
      real contrib = g2 + 1.0;
   end
```

```
    vout = s.contrib;                        // hierarchical read
end
```

Overriding a module parameter propagates into the block-scoped parameters that depend on it. The LRM's `#(.blk.p(4))` instance override of a block-scoped parameter is illegal (LRM 6.3.2) and rejected with a targeted diagnostic.

`paramset` blocks (a Verilog-AMS 3.4.6 feature adopted because real model libraries use them) work, including hidden-system-parameter assignments:

```
paramset fast_nmos nmos_va;
    .w = 2e-6; .l = 60e-9;
    .$mfactor = 4;
endparamset
```

3.3 Natures, disciplines, nets (LRM 3.5–3.7)

✓ Discipline/nature declarations with attribute access from expressions (`net.potential.abstol`); derived natures — `nature Xf : Flow` and deriving from a discipline's nature (`: electrical.flow`) — with full attribute inheritance; `ground` declarations in both orderings; vectored nets and ports with bit-selects; net initializers that become solver nodesets; `domain continuous`:

```
nature CurrentX : Current;    // derived nature, inherits abstol/units
    abstol = 1e-15;
endnature

electrical [3:0] bus;         // vectored net (range-then-name)
electrical data[0:7];         // vectored net (name-then-range, E-89)
electrical vout = 2.5;        // initializer -> OSDI nodeset (initial guess)
ground electrical gnd;
```

Both declaration orders are accepted for vectored nets and ports: the range-then-name form `electrical [3:0] bus;` / `input [3:0] bus;` and the name-then-range form `electrical bus[3:0];` / `input bus[3:0];` (E-89). A bit of a multi-bit input bus port reads its own terminal regardless of the port's position in the header — a bus port that is not the last port keeps its bits contiguous and in header order, so the simulator's positional terminal mapping stays correct (E-90). The name-then-range form also accepts a multi-name list with per-name widths (`input a[0:1], b[0:3], c;`, E-91). A net/port/array width may reference a parameter (`electrical [0:N-1] out;`, `real w[0:N-1];`, E-91): it is folded from the parameter's elaboration-time default and fixed at that value — a structural parameter, since the OSDI descriptor has one node count per module. Such a parameter is frozen to a **`localparam`** (E-92) so a netlist override cannot resize the compiled structure (or desync it from behavioural code); to change the width, edit the default and recompile. A multi-dimensional vectored port (`in[0:2][0:1]`) is not supported in either declaration order.

Signal-flow disciplines (potential-only or flow-only) are fully usable, including probe-only branches — probing a branch that is never contributed to reads its true flow (an ideal ammeter) instead of the 0-and-open-circuit a naive topology gives.

Named port branches (LRM 3.7.2) work as of E-84 and carry the same defining equation as a direct port-flow probe; contributing to one is a proper diagnostic (port branches are probe-only per the LRM):

```
branch (<p>) probe_p;          // named branch over port p
iprobe = I(probe_p);           // same value as I(<p>)
```

Hierarchical branch probes (E-86) read another instance's branches from outside it — named, unnamed, and port-branch forms, the last via a 0V ammeter synthesized at flattening so it reads that instance's own current rather than the node total:

```
V(top.a1.b)             // named branch of instance a1
V(top.d1.branch(va, vb)) // unnamed branch of instance d1
I(top.d1.branch(<p>))    // d1's own current into its port p
```

4. Expressions (LRM 4)

4.1 Operators and precedence (LRM 4.1, Table 4-2) — ✓

The full operator surface — arithmetic (incl. `**`), comparison, logical, bitwise, shifts (<</>> and arithmetic <<</>>>), ternary (including over strings), concatenation `{a,b}` and replication `{n{x}}` — was audited operator-by-operator and level-by-level against Table 4-2 with self-checking bitmask modules whose failure modes are numerically visible. Two real defects found and fixed in the audits: unary ~ emitted arithmetic negate, and the constant folder disagreed with the runtime on >>; one precedence defect: `%` bound tighter than `*`/`/` (so `6*7%4` evaluated to 18 instead of 2). Associativity verified including `2**3**2 = 64` (left-associative per Verilog) and unary binding above `**` (`-2**2 = 4`).

4.2 Built-in and environment functions (LRM 4.3, 9.7) — ✓

`ln`/`log`/`exp`/`sqrt`/`pow`/`min`/`max`/`abs`/`floor`/`ceil`, trigonometric and hyperbolic families (integer `min`/`max`/`abs` correctly integer-typed; integer division/rounding away from zero per the LRM), and the environment: `$temperature` (kelvin — verified as the exact kT/q law across a −50…150 °C sweep), `$vt` and `$vt(T)`, `$abstime`, `$realtime` (identical in the continuous-time analog context), `$simparam` and `$simparam$str`, `$param_given`, `$port_connected`, `$mfactor` and the position hidden parameters.

4.3 Analog operators (LRM 4.5) — ✓ (one documented deviation)

Every analog operator produces exact Jacobian contributions via autodiff. The full argument surface (trailing tolerances, optional arguments) was audited at runtime — compiling proves nothing about whether a trailing argument is honored.

```
// time derivative & integrals
I(p,n) <+ ddt(c * V(p,n));
phi = idtmod(K * V(vco), 0.0, 1.0);         // phase wrapping (VCO idiom)
x   = idt(V(in), 0.5, rst);                 // reset form holds at ic

// delays and filters
V(out) <+ absdelay(V(in), 1n);
V(out) <+ transition(sel ? 1.0 : 0.0, 0, 10n, 5n);
V(out) <+ slew(V(in), 1e6, -1e6);           // LRM negative max_neg_rate
V(out) <+ laplace_np(V(in), '{1.0}, '{-6.283e3, 0.0});  // complex pairs
V(out) <+ zi_nd(V(in), '{0.5, 0.5}, '{1.0}, 1u);

// small-signal & events
gm = ddx(Id, V(g,s));                       // symbolic derivative
tc = last_crossing(V(in) - 0.5, +1);
V(out) <+ ac_stim("ac", 1.0, 90.0);         // module AS the AC stimulus
```

- `ddt`, `idt` (with initial conditions that survive into transient, and the assert/reset forms stable enough for relaxation oscillators), `idtmod` (integrates unbounded internally, wraps the returned value), `ddx` with respect to potentials *and flows*: ✓
- `absdelay` (+maxdelay), `transition` (3/4/5-argument, `` `default_transition `` defaults, DC well-posed), `slew` (honoring the LRM's *negative* `max_neg_slew_rate` — the sign defect that made it ignore its input was found in the audit): ✓

- `laplace_nd/np/zd/zp` via exact state-space realization and `zi_nd/np/zd/zp` via bilinear transform, both with complex pole/zero pairs per the LRM's (re, im) vector convention and parameter-dependent coefficients: ✓
- `last_crossing` with simulator-side waveform history: ✓
- noise sources (LRM 4.6): `white_noise`, `flicker_noise`, `noise_table`, `noise_table_log` (inline or file data); operating-point-dependent factors and `ddt()`-shaped noise are exact with no extra matrix unknowns; same-named sources sum coherently per LRM 4.6.4 including anti-phase cancellation: ✓

  ```
  I(a,b) <+ white_noise(4.0 * `P_K * $temperature / r, "thermal");
  I(d,s) <+ gm * white_noise(gamma_4kT, "induced");   // op-dependent factor
  ```

- `analysis("ac","tran",…)` variadic, with the AC/noise operating point counting as its parent analysis per LRM 4.6.1: ✓
- `$limit` (built-in `"pnjlim"` and user functions — genuinely engages SPICE-style iteration limiting), `$bound_step`, `$discontinuity(n)` (clamps the next step *and* makes the integrator bisect onto the event): ✓
- `$table_model` (LRM 9.19 by clause, listed here as a modeling operator): 1-D piecewise-linear, N-dimensional multilinear, and natural cubic-spline interpolation — lowered to differentiable MIR so *all* partial derivatives feed the Jacobian (a table-based MOSFET gets exact gm and gds):

  ```
  I(d, s) <+ $table_model(V(g, s), V(d, s), "mos_iv.tbl", "1L");
  ```

- ⚠ **`limexp`** is implemented stateless (exact `exp` below the overflow threshold, tangent-continued above): a stateful previous-iterate limiting version was built and *reverted* because correct converged values under limiting require the simulator's limiting-RHS correction, which applies to circuit unknowns, not to `limexp`'s derived argument. `$limit` provides genuine iteration limiting. Documented decision with the failing evidence preserved.

Restrictions enforced as the LRM demands (4.5.1): analog operators inside loops or solution-dependent conditionals are rejected with diagnostics that name the actual construct and cite the clause.

5. Analog behavior (LRM 5)

5.1 Analog blocks (LRM 5.2, 6.2) — ✓

Multiple `analog` and `analog initial` blocks per module execute as-if-concatenated in source order — verified including across instance flattening and with declarations interleaved between blocks.

5.2 Contribution statements (LRM 5.6) — ✓

Direct contributions with accumulation semantics, switch branches (V-then-I), indirect branch assignment (the LRM's ideal-op-amp construct), and port-flow probes:

```
V(out): V(inp, inn) == 0;      // indirect: drive out so V(inp,inn)=0
I(p,n) <+ V(p,n)/r;            // accumulates with other <+ statements
iport = I(<p>);                // total flow through port p (ideal probe)
branch (<p>) pb;               // named port branch: I(pb) == I(<p>)  (E-84)
```

`$port_connected` works on connected *and* unconnected ports of flattened instances (resolved per instance at elaboration time), and instantiating an undefined module is a hard error rather than a silent drop (both E-84). Bus actuals may be part-selects — positional, named, or width-1 onto a scalar port (E-85):

```
adc2 hi (out[3:2], in);          // positional slice
adc2 lo (.out(out[1:0]), .in(r)); // named slice
```

5.3 Procedural statements (LRM 5.7–5.9) — ✓

`if`/`else`; `case` over integers, reals, strings, and arrays (element-wise); all four loops with runtime trip counts that honor model-card parameter overrides; `disable` as the LRM's early-exit (Verilog-A has no `break`/`continue`, and the compiler says so with the `disable` idiom in the diagnostic):

```
for (i = 0; i < n; i = i + 1) acc = acc + w[i] * x[i];
repeat (4) y = f(y);
do begin y = y/2; end while (y > 1);   // post-test: body runs once

begin : scan
    integer k;
    for (k = 0; k < 8; k = k + 1)
        if (v[k] > vth) disable scan;  // the break idiom
end
```

5.4 **casex** / **casez** — ✓

The don't-care case variants, with the 2-state semantics the analog subset implies: `x`/`z`/`?` digits of a based literal written directly as an item form a comparison mask — the arm matches on every *care* bit. `casex` treats x/z/? as don't-cares, `casez` only z/?. The classic priority-encoder idiom (from the committed `examples/casexz_examples/`):

```
casex (req)
    4'b1xxx: grant = 8;     // highest set bit wins
    4'b01xx: grant = 4;
    4'b001x: grant = 2;
    4'b0001: grant = 1;
    default: grant = 0;
endcase
```

Don't-care literals anywhere else, x digits under `casez`, and non-integer discriminants are located errors — no silent zeros.

5.5 Events (LRM 5.10) — ✓

`@(initial_step)` and `@(final_step)` genuinely gate (once at the start; once at the *successful* end, seeing the converged solution), with analysis-phase lists filtering by analysis type; `cross`/`above`/ `timer` with tolerances and persistent state; event OR lists:

```
@(initial_step("ac", "tran")) voff = 0.1;
@(cross(V(s) - vth, +1) or timer(0, 1u))
    n_events = n_events + 1;
@(final_step) $strobe("peak = %g at end of run", peak);
```

An operating point fires both step events (a single point is first and last); a failed analysis never fires `final_step`.

5.6 Analog functions (LRM 5.11) — ✓

Declared functions with input/output/inout arguments, arrays in all three roles plus array return values, locals (including array locals), loops inside bodies; inlined at the call site so derivatives flow through automatically:

```
analog function real dot;
    input real a[0:3], b[0:3];
    integer k;
    begin
        dot = 0;
        for (k = 0; k <= 3; k = k + 1)
            dot = dot + a[k]*b[k];
    end
endfunction
```

Recursion is illegal in Verilog-A; both direct and mutual recursion are clean errors naming the call cycle (mutual recursion formerly overflowed the compiler stack).

6. Hierarchy (LRM 6) — ✓

Module instantiation with named and positional ports and parameter overrides, open ports, instance arrays, arbitrary nesting, across `` `include `` boundaries; **`generate`** in all three forms with genvars usable in any expression; **`defparam`** with its LRM precedence over instance overrides; hierarchical names and `$root`; implicit nets; net concatenation in port connections:

```
// genvars usable in any expression, e.g. a parameter override
module gpexpr(a, c);
    inout a, c; electrical a, c;
    genvar i;
    generate
        for (i = 0; i < 3; i = i + 1) begin : g
            res #(.r(1e3*(i+1))) ri (a, c);   // 1k || 2k || 3k
        end
    endgenerate
endmodule

defparam u1.r = 2e3;                 // hierarchical override
defparam u1.u2.r = 4e3;              // two levels down; wins over #(...)
```

One boundary the OSDI target makes explicit (and the compiler explains in its diagnostic): structure binds at compile time — a `generate` condition or bound may depend on genvars and literals but not on module *parameters*, because parameters arrive from model cards at simulation time, after structure is frozen. Loop trip counts and `if` arms, being behavior, may depend on parameters freely. ⚠ documented deviation with rationale.

7. System tasks and functions (LRM 9)

7.1 Display (LRM 9.4) — ✓

All five kinds (`$strobe`, `$display`, `$write`, `$monitor`, `$debug`) with the complete format surface — `[flags][width][.precision]` on every conversion, dynamic ∗ widths, `%e/f/g/r` (engineering notation), `%d/h/o/b/c/s`, `%m`, `%%` — verified against exact printf output:

```
$strobe("Vth=%7.4f V  region=%2d  id=%r  name=%s", vth, reg, id, name);
```

7.2 File and string I/O (LRM 9.5, 9.8) — ✓

`$fopen/$fclose/$fdisplay/$fwrite/$fstrobe/$fmonitor/$fdebug/$fflush/ $ftell/$fseek/$rewind/$fe` and `$swrite/$sformat/$sscanf/ $fgets/$fscanf`:

```
integer fd;
analog @(final_step) begin
    fd = $fopen("iv.dat");
    $fdisplay(fd, "%g %g", vpeak, ipeak);
    $fclose(fd);
end
```

7.3 Random and distributions (LRM 9.13) — ✓ (deterministic semantics)

$random, $arandom, and every $dist_*/$rdist_* (uniform, normal, exponential, poisson, chi-square, t, erlang), verified against closed-form moments. ⚠ One deliberate semantic: draws are a deterministic function of *(seed, call site)* with no in-place seed advance — an advancing seed would return different values on every Newton iteration and destroy convergence. This gives reproducible Monte Carlo and independent per-instance variation, at the cost of in-evaluation sequences (which have no convergent meaning in an analog solver anyway).

```
parameter integer seed = 1;
real gain;
analog begin
    gain = 1.0 + sigma * $rdist_normal(seed, 0.0, 1.0);  // per-instance mismatch
    ...
end
```

7.4 Simulation control (LRM 9.6–9.7) — ✓

$finish, $stop, $fatal are honored with simulator-correct timing (at the accepted-point boundary; $finish fires @(final_step) and ends the analysis cleanly; $fatal during setup reads as a configuration rejection), and $warning/$error/$info display.

7.5 The mechanism-unavailable group — ⚠

$test$plusargs/$value$plusargs (ngspice has no plusargs), $simprobe, $analog_node_alias/$analog_port_a
compile and return their LRM-specified fallback values (false / supplied default / 0). This is the LRM's own escape hatch for hosts without the mechanism; models using them run predictably instead of being rejected.

7.6 Attributes as simulator metadata — ✓

(* desc="…" *) on a variable exposes it as an operating-point variable (print @n1[gm], .save, .meas); (* type="instance" *) on a parameter puts it on the instance line and under alter/.dc sweeps; desc/units on parameters populate the OSDI descriptor.

8. Compiler directives (LRM 10) — ✓

The predefined source-location macros work (E-85): `__FILE__ expands to the source file's basename, `__LINE__ to the exact line (a use inside a `define body reports the definition site — documented textual-expansion semantics):

```
$strobe("evaluated at %s:%0d", `__FILE__, `__LINE__);
```

`define with arguments (macros-in-macros, macro calls as arguments), `ifdef/`ifndef/`elsif/`else/`endif chained and nested, `include chains, `undef, `resetall, `default_transition, and the housekeeping set (`celldefine, `timescale, `line, `pragma, `default_nettype, …). Recursive macro expansion — direct or mutual — is a clean located error (formerly a compiler stack overflow), while the legal same-macro nesting inside *arguments* (`` `QUAD(x) = `TWICE(`TWICE(x)) ``) expands finitely:

```
`define TWICE(x) ((x)*2.0)
`define QUAD(x) (`TWICE(`TWICE(x)))   // same-macro nesting in ARGUMENTS: legal
`ifdef HIGH_PRECISION
    `define TOL 1e-12
`else
    `define TOL 1e-9
`endif
```

(`` `default_discipline `` is AMS-only and out of scope — implicit nets themselves are Verilog-A and work, taking their discipline from the connected port.)

9. Compliance summary

LRM area	Status	Verified by
2 Lexical (identifiers, numbers, strings, attributes)	✓	`escid`, `stresc`, `comment` suites
3.2–3.3 Value types, variables, persistence	✓	`vartype`, `variable_persistence`, `intstate`, `varinit`
3.4 Parameters (ranges, localparam, aliasparam, arrays, paramset, block-scoped)	✓ (default-exemption ⚠ documented)	`paramrange`, `localparam`, `array`, `paramset`, `paramsethsp`, `blockparam`
3.5–3.7 Natures, disciplines, nets, buses, nodesets	✓	`derivednature`, `domainbind`, `bus`, `netinit`, `ground`, `signalflow`
4.1–4.3 Operators, precedence, functions	✓ (audited)	`operator`, `precedence`, `shift`, `concat`
4.5 Analog operators (all)	✓ (`limexp` stateless ⚠ documented)	`absdelay`, `laplace`, `zi`, `slew`, `transition`, `idt*`, `ddx`, `opargs`, `discontinuity`, `last_crossing`
4.6 Noise (incl. correlation 4.6.4)	✓	`noise`, `noisecorr`, `noisejw`
5.2/6.2 Analog blocks (multiple)	✓	`multianalog`
5.6 Contributions, indirect assignment, port flow (incl. named port branches)	✓	`indirect_assignment`, `portflow`, `signalflow`, `lrm`
5.7–5.9 Procedural statements, loops, disable	✓	`analogloop`, `dowhile`, `repeat`, `disable`, `arraycase`
casex/casez	✓	`casexz`
5.10 Events (steps, cross/above/timer, OR lists, phase lists)	✓	`initial_step`, `finalstep`, `cross`, `timer`, `lrmcorner`
5.11 Analog functions (arrays in/out/return)	✓	`funcarray`, `arrayout`, `arrayret`
6 Hierarchy (instantiation, generate incl. legacy analog-block form, defparam, $root, part-select connections, hierarchical branch probes)	✓ (param-shaped structure ⚠ explained)	`instantiation`, `generate`, `legacygen`, `defparam`, `hiername`, `implicitnet`, `partselect`, `hierbranch`
9.4–9.8 Display, file/string I/O	✓	`display`, `fileio`, `stringio`

LRM area	Status	Verified by
9.13 Random/distributions	✓ (deterministic seed ⚠ documented)	`rng`, `montecarlo`
9 misc ($finish family, $simparam, attributes) plusargs / simprobe / node aliases	✓ ⚠ LRM fallbacks	`simctrl`, `simparamstr`, `opvar alias`
10 Compiler directives (incl. `` `__FILE__ ``/ `` `__LINE__ ``)	✓	`preproc`, `directive`, `defaulttransition`, `filemacro`
Mixed-signal / digital (outside Annex C)	× by scope	—

As of E-84 the standard's own examples are a compliance suite: every code example in the LRM 2023 PDF is extracted and compiled (`examples/lrm_examples/` — 40 compile, 19 documented limitations pinned to their diagnostics, 21 correctly rejected as mixed-signal), and the 146 non-module fragments double as a no-crash fuzz corpus.

Beyond construct-level checks, three cross-cutting guards pin that the *whole language implementation* produces correct physics: the 92-model industry corpus compiles and runs, the static/dynamic/thermal physics suites recover textbook device laws through the full pipeline (60 mV/dec, C $\equiv$ dQ/dV across analyses, E_g $\approx$ 1.25 eV from a compiled MEXTRAM), and the twin benchmarks show OSDI output matching hand-coded simulator devices to machine precision.